

Visual Mask System

技術說明文件

系統架構 · 模型解析 · 程式碼說明

Ethan / 獨立研究專題
2025

Chapter 1

連線架構：TCP/TLS → WebSocket 流程

本章以 TCP/TLS 三向交握的概念為基礎，說明 Visual Mask System 中 chat_server (FastAPI) 與 client (瀏覽器前端) 之間完整的連線建立、身份驗證與雙向通訊機制。

1.1 TCP/TLS 三向交握回顧

標準的 TCP+TLS 連線流程分為三個階段：

- TCP 三向交握 (SYN → SYN+ACK → ACK)：建立可靠的傳輸層連線
- TLS 握手 (憑證交換 → 金鑰協商)：在 TCP 之上建立加密通道，確認雙方身份
- HTTP/WebSocket 應用層：TLS 通道建立後，才開始傳送應用資料

本專案雖然在本機 (localhost) 執行不使用 TLS，但其連線流程在邏輯上與 TCP/TLS 完全對應，下表整理出這種對應關係：

TCP/TLS 步驟	本專案對應機制
TCP SYN / SYN-ACK / ACK	瀏覽器建立 TCP 連線至 localhost:8000 (FastAPI / Uvicorn)
TLS 憑證交換 (身份驗證)	POST /api/login — 前端送出帳號密碼，verify_login() 查詢 MySQL users 表
TLS 金鑰協商完成 → 安全通道建立	伺服器回傳 200 OK + {username}，前端取得已驗證的 username
HTTP Upgrade: websocket 請求	前端執行 new WebSocket('ws://localhost:8000/ws/{username}')
101 Switching Protocols	FastAPI @app.websocket 處理器呼叫 ws.accept()，ConnectionManager 儲存連線
全雙工 WebSocket 資料流	使用者傳訊息 → T5 推理 → AnimateDiff 生成 → broadcast 回前端

1.2 登入：身份驗證階段 (對應 TLS Handshake)

當使用者輸入帳號密碼後，前端發送 HTTP POST 請求至伺服器：

前端登入請求

```
# ■■■JavaScript■
const res = await fetch('/api/login', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({ account, password })
});
const data = await res.json(); // { message: "■■■■", username: "Alice" }
```

伺服器端 (chat_server.py) 接收並驗證：

伺服器登入端點

```
# chat_server.py
@app.post("/api/login")
async def login(req: LoginRequest):
    username = verify_login(req.account, req.password) # ■ MySQL
    if username is None:
        raise HTTPException(status_code=401, detail="■■■■■■■■")
    return {"message": "■■■■", "username": username}
```

[Note] 此步驟的語義等同於 TLS 中的「憑證驗證」——client 向 server 出示憑據 (帳密)，server 確認身份後才允許後續的升級連線。

1.3 WebSocket 升級 (對應 101 Switching Protocols)

身份驗證通過後，前端立即建立 WebSocket 連線。此步驟在 HTTP 層面發送 Upgrade: websocket 標頭，對應 TCP/TLS 流程中的應用層升級：

前端 WebSocket 升級

```
// ■■■JavaScript■■■■■ WebSocket
const ws = new WebSocket(`ws://localhost:8000/ws/${username}`);
ws.onopen = () => console.log("■■■■");
ws.onmessage = (e) => handleMessage(JSON.parse(e.data));
ws.onclose = () => console.log("■■■■");
```

伺服器端 WebSocket 接受

```
# chat_server.py - ■■■■■■■■■
@app.websocket("/ws/{client_name}")
async def websocket_endpoint(websocket: WebSocket, client_name: str):
    await manager.connect(websocket, client_name)
    # manager.connect() ■■■
    # 1. await ws.accept() → ■■ 101 Switching Protocols
    # 2. ■■■■■■■■■ → ■■■■ username ■■■■■■■■■
    # 3. connections.append(ws, name)
```

1.4 雙向通訊：兩階段廣播 (全雙工資料流)

連線建立後進入全雙工模式。本系統採用「兩階段廣播」設計，因為視覺生成 (AnimateDiff) 耗時較長 (1-2 分鐘) ，若等待完成後才回應，使用者體驗極差。因此拆成：

- Phase 1 (T5 推理完成，約 2-5 秒)：立即廣播 rationale + emotion label，前端顯示推理結果並開始轉圈等待圖片
- Phase 2 (AnimateDiff 生成完成，約 1-2 分鐘)：廣播含 mask_file 路徑的完整結果，前端渲染面具動畫

兩階段廣播機制

```
# chat_server.py - WebSocket
while True:
    text = await websocket.receive_text()
    msg_id = uuid.uuid4().hex # ID

    # Phase 1 T5 ~2-5s
    decision = reasoner.analyze(text, user_prior)
    decision["mask_id"] = get_user_active_lora(client_name)

    await manager.broadcast({
        "msg_id": msg_id,
        "sender": client_name,
        "text": text,
        "ai_rationale": decision["rationale"],
        "ai_label": decision["emotion"],
        "status": "generating", # 
    })

    # Phase 2 AnimatedDiff ~1-2min
    mask_file = await loop.run_in_executor(executor, orchestrator.run, decision)

    await manager.broadcast({
        "msg_id": msg_id,
        "mask_file": mask_file,
        "status": "done", # 
    })
```

[Note] msg_id 使用 UUID 而非 sender::text 作為 key，是為了解決同一使用者連續送出相同文字時，前端 DOM 元素 id 衝突導致面具覆蓋的 bug。

1.5 ConnectionManager：連線狀態管理

ConnectionManager 完整實作

```
class ConnectionManager:
    def __init__(self):
        self.connections: list[tuple[WebSocket, str]] = []

    async def connect(self, ws: WebSocket, name: str):
        await ws.accept()
        # 接收成功
        self.connections = [c for c in self.connections if c[1] != name]
        self.connections.append((ws, name))
        print(f"[{name}] 已連接")

    def disconnect(self, ws: WebSocket):
        self.connections = [c for c in self.connections if c[0] != ws]

    async def broadcast(self, payload: dict):
        text = json.dumps(payload, ensure_ascii=False)
        for ws, name in list(self.connections):
            try:
                await ws.send_text(text)
            except Exception:
                # 發送失敗 → 從列表中移除
                self.connections = [c for c in self.connections if c[0] != ws]
```

Chapter 2

TextSummarizer 與 BART-large-CNN

TextSummarizer 是系統 Input Layer 的第一個模組，負責將原始使用者輸入壓縮為顯著語意 token (salient tokens)，再傳入後續的 M_ICL_Integrator 與 T5 推理模組。它底層使用 Facebook 的 BART-large-CNN 預訓練模型。

2.1 程式碼詳解

layers/input/text_summarizer.py

```
import torch
from transformers import BartForConditionalGeneration, BartTokenizerFast

class TextSummarizer:
    def __init__(self, model_name="facebook/bart-large-cnn", device=None):
        device_id = device if device is not None else (0 if torch.cuda.is_available() else -1)
        self._device = "cpu" if device_id < 0 else f"cuda:{device_id}"

        self._tokenizer = BartTokenizerFast.from_pretrained(model_name)
        self._model = BartForConditionalGeneration.from_pretrained(model_name).to(self._device)
        self._model.eval()

    def extract_salient_tokens(self, text: str,
                              max_length: int = 50,
                              min_length: int = 10) -> str:
        inputs = self._tokenizer(
            text, return_tensors="pt", max_length=1024, truncation=True
        ).to(self._device)

        with torch.no_grad():
            ids = self._model.generate(
                inputs["input_ids"],
                max_length=max_length,
                min_length=min_length,
                length_penalty=2.0,      # ■■■■■■
                num_beams=4,             # Beam Search ■4 ■■■■
                early_stopping=True,
            )
        return self._tokenizer.decode(ids[0], skip_special_tokens=True)
```

逐行重點說明

- device 自動偵測：若有 CUDA GPU 則使用 GPU；本專案中 BART 鎖定 CPU (device=-1)，GPU 保留給 AnimateDiff 視覺生成使用
- BartTokenizerFast：快速版 Tokenizer，使用 Rust 實作，速度較慢版快約 10 倍

- `model.eval()`：關閉 Dropout、BatchNorm 等訓練模式層，確保推理時輸出穩定
- `torch.no_grad()`：不計算梯度，節省記憶體與運算
- `length_penalty=2.0`：值 > 1 鼓勵模型生成較長的摘要序列
- `num_beams=4`：Beam Search 同時追蹤 4 條解碼路徑，選擇整體機率最高的序列
- `max_length=50`,
`min_length=10`：輸出摘要的字元長度限制，本模組專注於「關鍵語意提取」

2.2 BART 模型架構

BART (Bidirectional and Auto-Regressive Transformers) 由 Facebook AI Research 於 2019 年提出 (論文：Lewis et al., 2019)。它是一種 Encoder-Decoder 架構的預訓練語言模型，整合了 BERT 的雙向理解能力與 GPT 的自回歸生成能力。

組件	說明
Encoder (編碼器)	雙向 Transformer，一次讀取完整輸入序列 (類似 BERT)。每個 token 的表示都能看到前後文，適合理解語義
Decoder (解碼器)	左到右自回歸 Transformer (類似 GPT)。生成每個 token 時，只能看到已生成的前綴與 Encoder 輸出，確保生成合理
Cross-Attention	Decoder 的每一層都有 Cross-Attention 層，允許 Decoder 在生成時參照 Encoder 的完整輸出 (全文理解)
位置編碼	學習式位置嵌入 (Learned Positional Embedding)，最大輸入長度 1024 tokens

2.3 BART 預訓練策略

BART 的預訓練目標是：給定被損壞 (corrupted) 的文字，重建原始文字。研究者嘗試了多種文字損壞策略，以下是主要方法：

- Token Masking：隨機用 [MASK] 替換部分 token (類似 BERT 的 MLM)
- Token Deletion：隨機刪除 token，模型必須判斷被刪除的位置
- Text Infilling (最有效)：將連續片段替換為單一 [MASK]，模型需預測整段缺失文字，學習到更豐富的語言結構
- Sentence Permutation：打亂句子順序，模型學習語篇連貫性
- Document Rotation：循環移位文件起始點，訓練模型辨識正確的文件開頭

[Note] BART-large 使用約 400M 參數，在 Books + CC-News + OpenWebText + Stories 等語料上預訓練

2.4 CNN/Daily Mail 微調 (Abstractive Summarization)

BART-large-CNN 是在 CNN/Daily Mail 資料集上進一步微調的版本，專門用於新聞文章的抽象式摘要生成。與擷取式摘要 (Extractive

Summarization) 不同，抽象式摘要會生成原文中不存在的新句子，能更自然地表達核心要義。

比較面向	說明
Extractive (擷取式)	直接從原文挑出重要句子組成摘要，不生成新詞彙。速度快但可能不連貫
Abstractive (抽象式，BA RT)	生成全新句子表達核心語意，更自然流暢，但需要強大的語言生成能力
CNN/Daily Mail 資料集	約 28 萬篇新聞文章 + 對應的人工摘要 (highlights)，是 NLP 摘要任務最常用的 benchmark

2.5 在本系統中的角色

在 Visual Mask System 的資料流中，TextSummarizer 處於最前端：

完整推理鏈資料流

```
# reasoning_engine.py - MCoTReasoningEngine.analyze() ████████

# Step 1: █████ → █████ token
salient = self.summarizer.extract_salient_tokens(user_input)
#   Input:  "██████████████████████████████████████"
#   Output: "worked late, boss scolding, feeling overwhelmed"

# Step 2: █████████ Context ██
history_ctx = self.integrator.synthesize(salient, history)

# Step 3: ██ T5 █████
raw_context = (
    f"Context: {history_ctx}. "
    f"Current Stream: {salient}. "
    f"User Prior: {user_prior}"
)

# Step 4: T5 Stage 1 █████rationale█
rationale = self.decoder.decode(raw_context)

# Step 5: T5 Stage 2 █████emotion label█
label = self.analyzer.analyze(raw_context, rationale)
```

Chapter 3

M_ICL_Integrator：多輪對話情緒趨勢整合

M_ICL_Integrator (Multi-shot In-Context Learning Integrator) 是 Input Layer 的第三個模組，負責：

- 整合 BART 摘要結果與對話歷史，生成 Context 字串
- 追蹤多輪對話中的情緒強度變化趨勢
- 將趨勢資訊附加在 Context 中，讓 T5 推理時能感知情緒走向

3.1 完整程式碼

```

from collections import Counter

_EMOTION_INTENSITY: dict[str, float] = {
    "calm": 0.10, "bored": 0.20, "neutral": 0.30,
    "happy": 0.40, "joyful": 0.40,
    "sad": 0.50, "disappoint": 0.52, "passive": 0.55,
    "withdraw": 0.58, "sarcastic": 0.60,
    "anxious": 0.62, "confused": 0.62,
    "frustrat": 0.72, "stressed": 0.72, "overwhelm": 0.74,
    "angry": 0.85, "disgusted": 0.80, "fearful": 0.82,
    "furious": 0.95, "rage": 0.95,
}

_ESCALATE_THRESH = 0.20 # delta >= 0.20 → escalating
_DEESCALATE_THRESH = -0.20 # delta <= -0.20 → de-escalating

def _to_intensity(label: str) -> float:
    label_lower = label.lower()
    for key, val in _EMOTION_INTENSITY.items():
        if key in label_lower:
            return val
    return 0.50 # ■■■■■■■■

class M_ICL_Integrator:
    def synthesize(self, salient_tokens: str, history: list) -> str:
        history_str = (
            " | ".join(f"{m['role']}: {m['content']}" for m in history)
            or "Initial state."
        )
        trend = self._analyze_emotion_trend(history)
        if trend == "stable":
            return history_str
        return f"{history_str}. Emotional trajectory: {trend}"

    def _analyze_emotion_trend(self, history: list) -> str:
        emotion_seq = [m["content"] for m in history if m["role"] == "System"]
        if len(emotion_seq) < 2:
            return "stable"
        intensities = [_to_intensity(e) for e in emotion_seq]
        mid = max(len(intensities) // 2, 1)
        avg_early = sum(intensities[:mid]) / mid
        avg_late = sum(intensities[mid:]) / max(len(intensities) - mid, 1)
        delta = avg_late - avg_early
        if delta >= _ESCALATE_THRESH:
            return f"escalating toward {emotion_seq[-1]}"
        if delta <= _DEESCALATE_THRESH:
            return f"de-escalating toward {emotion_seq[-1]}"
        dominant = Counter(emotion_seq).most_common(1)[0][0]
        return f"stable at {dominant}"

```

3.2 情緒強度映射表 (_EMOTION_INTENSITY)

`_EMOTION_INTENSITY` 將情緒標籤對應到 0.0–1.0 的強度數值，用於趨勢計算。注意 key 使用部分字串 (partial match)，例如 'frustrat' 同時匹配 'frustrated' 和 'frustrating'。

	對應情緒	說明
	calm, bored, neutral	低喚起 (Low Arousal) 正向或中性情緒
	happy, joyful, sad, disappoint, passive	中低強度情緒
	sarcastic, anxious, confused, frustrated, stressed	中高強度負向情緒
	angry, disgusted, fearful, furious, rage	高強度負向激動情緒

3.3 情緒趨勢偵測邏輯

`_analyze_emotion_trend()` 從 `ContextualBuffer` 中取出 `System` 角色的情緒標籤序列，計算前段與後段平均強度差 (`delta`)，判斷情緒走向：

- `delta >= +0.20` (`escalating`)：情緒強度明顯上升，在 `Context` 中附加 'Emotional trajectory: escalating toward {最新情緒}'
- `delta <= -0.20` (`de-escalating`)：情緒強度明顯下降，附加 'Emotional trajectory: de-escalating toward {最新情緒}'
- 其餘 (`stable`)：情緒相對穩定，附加 'stable at {主要情緒}' 或直接不附加

[Note] 此趨勢資訊直接嵌入 T5 的 `Context` 欄位，讓模型在推理時能考量情緒的動態變化，而不只是當前一則訊息的內容。

3.4 與 `ContextualBuffer` 的協作

`ContextualBuffer` 與 `M_ICL_Integrator` 協作示例

```
# reasoning_engine.py ████████

# ContextualBuffer █████ 5 █████User / System ████
self.buffer = ContextualBuffer(buffer_size=5)

# ██████████ buffer
self.buffer.add_message("User", user_input) # ████████
self.buffer.add_message("System", label) # ██████████

# M_ICL_Integrator █ buffer ██████████
history = self.buffer.get_history()
history_ctx = self.integrator.synthesize(salient, history)
# ████████
# "User: ███ | System: sad | User: █████ | System: frustrated.
# Emotional trajectory: escalating toward frustrated"
```

Chapter 4

SemanticIntentDecoder : M-CoT Stage 1 社交語境推理

SemanticIntentDecoder 是 Multimodal Chain-of-Thought (M-CoT) 兩階段推理管線的 Stage 1 模組。它接收結構化的 context 字串，生成描述使用者社交意圖與情緒語境的 rationale (推理鏈) 文字，再交由 Stage 2 (AffectiveStateAnalyzer) 做最終情緒分類。

4.1 完整程式碼

layers/reasoning/semantic_decoder.py

```
import torch
from transformers import T5Tokenizer, T5ForConditionalGeneration

class SemanticIntentDecoder:
    """
    T5 Stage 1 – Social Reasoning
    ■■■raw_context■■■■■■■■■■user prior■
    ■■■rationale■■■■■
    """
    def __init__(self, model_path: str, device: str = "cpu"):
        self.device = device
        self.tokenizer = T5Tokenizer.from_pretrained(model_path)
        self.model = T5ForConditionalGeneration.from_pretrained(model_path).to(device)

    def decode(self, raw_context: str) -> str:
        input_text = f"social reasoning: {raw_context}"
        inputs = self.tokenizer(
            input_text, return_tensors="pt", truncation=True, max_length=256
        ).to(self.device)
        with torch.no_grad():
            outputs = self.model.generate(
                **inputs, max_length=150, num_beams=4, early_stopping=True
            )
        return self.tokenizer.decode(outputs[0], skip_special_tokens=True)
```

4.2 模型基底 : flan-t5-base

SemanticIntentDecoder 使用 Google 的 flan-t5-base 作為基底模型，在合成資料集 (train_stage1_synthetic.jsonl) 上微調而成。

項目	說明
基底模型	google/flan-t5-base (250M 參數)
架構	Encoder-Decoder Transformer (T5 系列)
任務前綴 (Task Prefix)	"social reasoning: " — T5 以任務前綴區分不同任務
輸入長度	max_length=256 tokens
輸出長度	max_length=150 tokens (rationale 為多詞說明文字)
解碼策略	Beam Search · num_beams=4
微調資料	合成的社交語境推理資料集 (多種情境、情緒)
checkpoint 路徑	models/mcot_rationale_model/

4.3 T5 Task Prefix 機制

T5 (Text-to-Text Transfer Transformer) 的設計哲學是將所有 NLP 任務統一為文字轉文字的問題。不同任務用不同的前綴字串 (task prefix) 加以區分，讓同一個模型能在不同任務間切換。

- "social reasoning: " → Stage 1：輸入對話語境，輸出情緒推理說明 (rationale)
- "emotion inference: " → Stage 2：輸入對話語境 + rationale，輸出情緒標籤
- "summarize: " → 原始 T5 摘要任務 (本專案用 BART，但 T5 也支援)

[Note] 兩個 Stage 的模型都從 flan-t5-base 微調，因此共用同一份 Tokenizer，在 reasoning_engine.py 中以 self.decoder.tokenizer 傳入 AffectiveStateAnalyzer

4.4 輸入 / 輸出格式範例

Stage 1 輸入/輸出格式示例

```
# raw_context T5 Stage 1
input_text = (
    "social reasoning: "
    "Context: User:  | System: sad | "
    "Emotional trajectory: escalating toward frustrated. "
    "Current Stream:  "
    "User Prior: A normal user."
)

# Stage 1 rationale
rationale = (
    "[Context Analysis]: User has been expressing fatigue over multiple turns, "
    "with an escalating emotional trajectory. "
    "[Social Inference]: The mention of overtime work and boss criticism indicates "
    "external stressors causing emotional distress. "
    "[Final Inference]: The user is likely experiencing frustration and stress."
)
```

Chapter 5

AffectiveStateAnalyzer : M-CoT Stage 2 情緒標籤推理

AffectiveStateAnalyzer 是 M-CoT 兩階段推理管線的 Stage 2 模組，也是整個推理鏈的最終輸出節點。它接收 Stage 1 產出的 rationale 加上原始 context，生成一個具體的情緒標籤（如 angry、calm、frustrated 等），供後續的 VisualInstructionGenerator 轉換為視覺提示詞，驅動 AnimateDiff 生成對應的面具動畫。

5.1 完整程式碼

layers/reasoning/affective_analyzer.py

```
import torch
from transformers import T5Tokenizer, T5ForConditionalGeneration

class AffectiveStateAnalyzer:
    """
    T5 Stage 2 – Emotion Inference
    ■■■raw_context + rationale
    ■■■emotion label■■■■■■■■ angry / sarcastic / neutral■
    """
    def __init__(self, model_path: str, tokenizer: T5Tokenizer, device: str = "cpu"):
        self.device = device
        self.tokenizer = tokenizer # ■ SemanticDecoder ■■■■■ tokenizer
        self.model = T5ForConditionalGeneration.from_pretrained(model_path).to(device)

    def analyze(self, raw_context: str, rationale: str) -> str:
        input_text = f"emotion inference: {raw_context} Rationale: {rationale}"
        inputs = self.tokenizer(
            input_text, return_tensors="pt", truncation=True, max_length=256
        ).to(self.device)
        with torch.no_grad():
            outputs = self.model.generate(
                **inputs, max_length=20, num_beams=4, early_stopping=True
            )
        return self.tokenizer.decode(outputs[0], skip_special_tokens=True).strip().lower
    ()
```

5.2 關鍵設計決策

設計項目	說明
共用 Tokenizer	Stage 1 與 Stage 2 共用同一份 T5Tokenizer (兩個模型都從 flan-t5-base 微調)，由 reasoning_engine.py 傳入，避免重複載入佔用記憶體
max_length=20	情緒標籤只有一個詞 (angry, calm 等)，輸出長度極短。相較 Stage 1 的 max_length=150，大幅節省解碼時間
.strip().lower()	正規化輸出，防止大小寫差異 (模型有時輸出 Angry 或 ANGRY)，確保與 _EMOTION_INTENSITY 映射表的 key 一致
任務前綴 'emotion inference: '	與 Stage 1 的 'social reasoning: ' 區分，讓微調後的模型知道此任務是情緒分類而非生成說明
無 Fallback 機制 (設計取捨)	直接回傳 .lower() 的輸出，若輸出不在已知情緒標籤中，由上層 (reasoning_engine 或 VisualInstructionGenerator) 處理 fallback

5.3 Stage 1 □ Stage 2 的 Chain-of-Thought 關係

M-CoT (Multimodal Chain-of-Thought) 的核心思想是：先讓模型「思考」 (Stage 1 生成 rationale)，再根據思考結果「決策」 (Stage 2 輸出 label)。這種兩步驟設計比直接預測 label 效果更好，因為：

- Stage 1 強迫模型明確化「為什麼是這個情緒」的推理過程，減少黑箱效應
- Stage 2 看到的輸入比 Stage 1 更豐富 (context + rationale)，能做出更準確的最終分類決策
- 兩個模型分開訓練，Stage 1 專注「生成說明文字」，Stage 2 專注「情緒分類」，任務分工明確

M-CoT 完整推理鏈 (reasoning_engine.py)

```
# reasoning_engine.py - ■■ M-CoT ■■■■■■
raw_context = (
    f"Context: {history_ctx}. "
    f"Current Stream: {salient}. "
    f"User Prior: {user_prior}"
)

# Stage 1■■■■■ → rationale
rationale = self.decoder.decode(raw_context)
# ■■[Context Analysis]: ... [Social Inference]: ... [Final Inference]: ...

# Stage 2■■■■■ → label
label = self.analyzer.analyze(raw_context, rationale)
# ■■frustrated

# Stage 3■■ T5■■■■■ → ■■■■■ + ■■
visual_prompt, intensity = self.vig.generate(label)
# ■■visual_prompt="dark stormy clouds, deep red aura..."
#     intensity=0.72
```

5.4 支援的情緒標籤

AffectiveStateAnalyzer 可輸出以下情緒標籤 (按強度由低到高排列) :

強度等級	情緒標籤
極低 (Calm)	calm, bored
低 (Positive)	neutral, happy, joyful
中 (Mild Negative)	sad, disappoint, passive, withdraw, confused
中高 (Stressed)	sarcastic, anxious, frustrated, stressed, overwhelmed
高 (Intense)	angry, disgusted, fearful
極高 (Extreme)	furious, rage